Plan de memorización a largo plazo

El objetivo no es pasar un examen la semana que viene. Es que en una entrevista de marzo de 2027, o en tu trabajo dentro de dos años, puedas explicar qué hace el CLR igual de claro que hoy. Este plan está hecho para eso.

Por qué olvidamos y cómo evitarlo

En 1885 el psicólogo Hermann Ebbinghaus midió cuánto olvidamos con el tiempo. Lo que encontró se llama **curva del olvido**: sin repasar, en un día se pierde más de la mitad de lo aprendido y en una semana cerca de tres cuartas partes. Pero cada vez que **recuerdas algo justo cuando estabas por olvidarlo**, la curva se vuelve más plana y el siguiente olvido tarda más.

De ahí salen las tres reglas de este plan:

1. **Recordar, no releer.** Releer da la sensación de que ya lo sabes. Lo que fija la memoria es el esfuerzo de sacar la respuesta sin mirar.
2. **Espaciar.** Repasar poco y seguido, con intervalos cada vez más largos: 1 día, 3 días, 1 semana, 3 semanas, 2 meses, 4 meses.
3. **Entender antes de memorizar.** Una tarjeta que no entiendes se olvida aunque la repitas. Por eso cada concepto pasa primero por la escalera de comprensión.

La escalera de comprensión

Un concepto no está "aprendido" hasta que llega al nivel 4. Cada concepto empieza en el 1.

Nivel	Qué puedes hacer	Cómo lo compruebas
1. Recordar	Decir la definición	La tarjeta de repaso te sale sin mirar
2. Explicar	Explicarlo con tus palabras y una analogía	2 minutos en voz alta, sin trabarte (técnica Feynman)
3. Aplicar	Usarlo o verlo en algo real	Lo comprobaste en tu máquina o en tu código
4. Conectar	Relacionarlo con otros conceptos y con DiSí	Puedes contar la historia completa donde aparece

Las calificaciones de 14 Evaluaciones te dicen en qué nivel está cada concepto. Un 6 de 10 normalmente significa nivel 1: sabes la definición pero no todas sus partes.

El sistema: tarjetas en Obsidian

Paso 1. Instalar el plugin (5 min, en la PC y en el celular)

1. En Obsidian: **Configuración → Plugins de la comunidad**. Si dice "Modo restringido", desactívalo.
2. **Explorar** → busca **Spaced Repetition** (autor: Stephen Mwangi, versión 1.15 o mayor) → **Instalar** → **Activar**.
3. Repite lo mismo en el celular. La configuración de los plugins no viaja por Git (la carpeta `.obsidian/` está excluida a propósito), así que cada dispositivo se configura una vez.

Paso 2. Configurar (5 min, en cada dispositivo)

Configuración → Spaced Repetition. Solo cambia lo que dice **Cambiar**; lo demás déjalo como viene. Si tu Obsidian está en español, el plugin también lo estará: te pongo los dos nombres.

Opción (inglés / español)	Valor	Por qué
Flashcard tags / Etiquetas de las tarjetas de memorización	`#flashcards`	Ya viene así. Encuentra todas tus notas con `#flashcards/…`
Convert folders to decks / ¿Convertir directorios a mazos?	Desactivado	Ya viene así. Los mazos salen de las etiquetas, no de carpetas
Convert ==highlights== to clozes / ¿Convertir ==resaltados== a deletreo de huecos?	**Cambiar: desactivado**	Si queda activo, cualquier texto entre `==` se vuelve tarjeta. En la teoría hay una tabla con el operador `==` de C# que crearía una tarjeta falsa
Convert **bolded text** to clozes / ¿Convertir texto en negrita…?	Desactivado	Ya viene así. Tus notas usan muchas negritas; si se activa, cada una sería una tarjeta
Separator for inline flashcards / Separador de tarjetas en línea	`::`	Ya viene así. Es el formato de todas tus tarjetas
Algorithm / Algoritmo	**SM-2-OSR**	Ya viene así. Es el algoritmo clásico de repetición espaciada (el de Anki), probado por años. El plugin también ofrece **FSRS**, más moderno, pero el propio plugin advierte que todavía está poco probado y puede perder datos al cambiar: **no lo actives**
Save scheduling comment on the same line / ¿Guardar el comentario de programación en la última línea?	**Cambiar: activado**	Así la fecha de repaso queda al final de la misma línea de la tarjeta: cada tarjeta sigue ocupando una sola línea y las notas se ven más limpias
Show context in cards	Activado	Ya viene así. Muestra de qué nota y sección sale la tarjeta
Show interval in review buttons	Activado	Ya viene así. Ves en cuántos días volverá la tarjeta según el botón que elijas
Status bar	Activado	Ya viene así. Abajo a la derecha verás cuántas tarjetas te tocan hoy

Paso 3. Comprobar que lee tus tarjetas (2 min)

Abre la paleta de comandos → **Spaced Repetition: Review flashcards from all notes** (en español: **Revisar tarjetas de todas las notas**). **No empieces a repasar todavía.** Primero revisa que aparezca este árbol de mazos:

```
flashcards
├── csharp
│   ├── s1-plataforma-net          17  (+ sus submazos = 38)
│   │   ├── errores                 9
│   │   └── dudas                  12
│   ├── s2-anatomia-proyecto       21  (+ sus submazos = 38)
│   │   ├── errores                 6
│   │   └── dudas                  11
│   ├── s3-tipos                   26
│   ├── s4-control-flujo           22
│   ├── s5-double-decimal          14
│   ├── s6-metodos                 18
```

```
│   ├── s7-convenciones              12
│   └── lab-primer-proyecto          15  (+ sus submazos = 27)
│       ├── errores                   4
│       └── dudas                     8
└── fundamentos
    └── 0-1-datos                    17
                          Total:    212
```

Si los números no coinciden, avísame con una captura antes de repasar.

Los mazos: uno por unidad de estudio

Regla de oro: solo repasas los mazos de lo que ya estudiaste. Por eso los mazos no son por área en general, sino por **unidad**: lo que estudias en una sesión (una sección de la teoría, una clase, un laboratorio).

Cada tarjeta empieza con la etiqueta de su mazo, en la misma línea:

`#flashcards/csharp/s1-plataforma-net ¿Qué es el JIT?::El compilador del CLR que traduce IL a código máquina…`

Al repasar, la etiqueta no se ve: solo la pregunta. Los errores y las dudas de una unidad son **submazos** de esa unidad (`…/errores`, `…/dudas`): cuando abres la unidad, el plugin incluye sus submazos.

Mazo	Unidad	Tarjetas en
`flashcards/fundamentos/0-1-datos`	Clase 0.1 Datos	11 Clase 0.1 Datos
`flashcards/csharp/s1-plataforma-net`	Teoría de C#, sección 1	10 Teoria de CSharp
`flashcards/csharp/s1-plataforma-net/errores`	Lo que fallaste de la sección 1	14 Evaluaciones
`flashcards/csharp/s1-plataforma-net/dudas`	Lo que preguntaste al leer la sección 1	15 Dudas resueltas
`flashcards/csharp/s2-anatomia-proyecto`	Teoría de C#, sección 2	10 Teoria de CSharp
`flashcards/csharp/s2-anatomia-proyecto/errores`	Lo que fallaste de la sección 2	14 Evaluaciones
`flashcards/csharp/s2-anatomia-proyecto/dudas`	Lo que preguntaste al leer la sección 2	15 Dudas resueltas
`flashcards/csharp/s3-tipos`	Teoría de C#, sección 3	10 Teoria de CSharp
`flashcards/csharp/s4-control-flujo`	Teoría de C#, sección 4	10 Teoria de CSharp
`flashcards/csharp/s5-double-decimal`	Teoría de C#, sección 5	10 Teoria de CSharp
`flashcards/csharp/s6-metodos`	Teoría de C#, sección 6	10 Teoria de CSharp
`flashcards/csharp/s7-convenciones`	Teoría de C#, sección 7	10 Teoria de CSharp
`flashcards/csharp/lab-primer-proyecto`	Laboratorio del 6 oct	12 Clase Primer proyecto por CLI

Nombres de las unidades que siguen: `csharp/s2-anatomia-proyecto`, `csharp/s3-tipos`, `fundamentos/0-1-texto`, `fundamentos/0-2-hardware`, etc. Cada unidad nueva se agrega a esta tabla y al registro de unidades.

Cómo repasar

1. Paleta de comandos → **Review flashcards from all notes** (**Revisar tarjetas de todas las notas**), o el ícono de tarjetas en la barra lateral.

2. Lee la pregunta y **contesta en voz alta, completa**, antes de mostrar la respuesta. Si la pregunta tiene dos partes, contesta las dos.
3. Muestra la respuesta y califícate con honestidad:

En el celular los botones no muestran el nombre, sino **en cuánto tiempo volverá la tarjeta**, por ejemplo `1m · 1d · 3d · 4d`. Van siempre en este orden, de izquierda a derecha:

Botón	Color	Cuándo usarlo
Again	Rojo (el de minutos)	No salió, o salió mal. La vuelve a mostrar en la misma sesión
Hard / Difícil	Naranja	Salió incompleta o te costó mucho
Good / Bien	Azul	Salió completa, con algo de esfuerzo. Debe ser tu respuesta más común
Easy / Fácil	Verde	Salió completa e inmediata

4. Al calificar, el plugin escribe al final de la línea de la tarjeta un comentario invisible como ``: fecha del siguiente repaso, intervalo en días y facilidad. **No lo borres:** es la memoria del plugin.

El repaso y Git (importante)

Como las fechas de repaso se guardan **dentro de las notas**, si repasas en el celular y en la PC el mismo día sin sincronizar, Git puede marcar un conflicto. Para evitarlo:

1. **Antes de repasar:** paleta de comandos → **Git: Pull**.
2. **Repasa en un solo dispositivo por día.**
3. **Al terminar:** paleta de comandos → **Git: Commit-and-sync**.

Si aun así aparece un conflicto, no lo resuelvas a mano: avísame y lo arreglo desde el repositorio.

En el celular, además:

- Desactiva el **Auto commit-and-sync** automático del plugin Git (déjalo en 0) y sincroniza solo a mano, al terminar de repasar. El 19 oct un respaldo automático subió versiones viejas de varias notas encima de las nuevas.
- **Antes** de cada Pull o Commit-and-sync, **cierra todas las notas abiertas**. Si una nota está abierta mientras se descargan cambios, Obsidian puede volver a guardar encima la versión vieja que tenía en pantalla (pasó el 19 oct con `10 Teoria de CSharp`). Si después del sync aparece un cambio que no hiciste, descártalo con ↶.
- Después de cada **Git: Pull**, **cierra Obsidian por completo y vuelve a abrirlo** antes de repasar. Así el plugin de tarjetas lee las notas nuevas y no escribe encima con las que tenía en memoria.

Cómo elegir qué mazo repasar

En la ventana de repaso aparece el árbol de mazos con el número de tarjetas pendientes. **Haz clic en el mazo de la unidad**, no en "flashcards", para repasar solo esa unidad. Cuando ya estudiaste varias unidades, haz clic en el mazo de arriba (`flashcards`) y repasas todo lo que te toque ese día.

Las tarjetas de una unidad que todavía no estudias se quedan quietas: el plugin no las cuenta como atrasadas hasta que las repases por primera vez.

Cómo escribir tus propias tarjetas

Las mejores tarjetas son las que escribes tú. Reglas:

- **Una idea por tarjeta.** "¿Qué trae el SDK?" es buena. "Explica el SDK, el runtime y el CLR" son tres tarjetas.
- **Con tus palabras**, no copiadas de la nota.
- **Preguntas de "por qué"**, además de "qué": "¿Por qué el dinero va en `decimal`?" vale más que "¿Qué es `decimal`?".
- **Cada error se vuelve tarjeta.** Lo que fallaste en una evaluación es exactamente lo que más necesitas repasar.
- **Si una tarjeta te falla tres veces seguidas, no la repitas más: reescríbela** o divídela. El problema es la tarjeta o que falta entender algo.

Pregunta	Respuesta
Formato: `#flashcards/<área>/ pregunta	respuesta`, una tarjeta por línea.

Cada unidad nueva: el protocolo

Así se trata **cada** cosa que estudias, desde el día en que la estudias. Las tarjetas las maneja el plugin; lo demás lo haces tú.

Día	Qué haces	Tiempo
Día 0 (estudias)	Estudias la unidad. Al final, 2 minutos de Feynman en voz alta	La sesión
Día 1	1. **Volcado de memoria** sin mirar. 2. Comparas con la nota y marcas lo que faltó. 3. Relees **solo** lo que faltó. 4. **Primer repaso** del mazo de la unidad	30–40 min
Día 2	Tarjetas que te toquen (el plugin decide) + cuentas la unidad en voz alta 3 min	10 min
Día 4 aprox.	Tarjetas + Feynman de los 2 conceptos que más te cuesten	10 min
Día 8 aprox.	**Evaluación:** contestas por escrito las preguntas de repaso de la unidad, sin mirar. Claude te califica en 14 Evaluaciones y compara con la anterior	20 min
Día 21 en adelante	Solo tarjetas (el plugin espacia cada vez más) + el examen acumulado del mes	—

¿Releer o no releer?

Al día siguiente, primero intenta recordar y solo después relee. Releer de entrada se siente útil porque todo te suena conocido, pero "me suena" no es "lo sé explicar". Si primero intentas sacar la información y fallas, el esfuerzo marca exactamente qué te falta, y la relectura posterior de *solo esas partes* se queda mucho mejor.

Y que al día siguiente "casi no recuerdes nada" es **normal**: es la curva del olvido funcionando. No significa que estudiaste mal. Significa que hoy toca el primer repaso.

Ejemplo: la sección 1 de C#, estudiada el 14 oct

Este es el plan concreto para lo que estudiaste ayer. Unidad: **sección 1 de 10 Teoria de CSharp**, mazo `csharp/s1-plataforma-net` (38 tarjetas con errores y dudas).

Hoy, jueves 15 oct (día 1, unos 40 min)

1. **Volcado de memoria (10 min), sin mirar nada.** En una hoja en blanco dibuja el recorrido de tu código, de `Program.cs` hasta la CPU, y escribe junto a cada paso lo que recuerdes. Usa como guía estas 8 palabras: *compilar, IL, JIT, runtime, SDK, CLI, CLR, DLL*. Si de una no recuerdas nada, déjala en blanco: está bien.
2. **Comparar (5 min).** Abre la sección 1 y marca con otro color lo que te faltó o tenías mal.
3. **Releer solo lo marcado (10 min).** No toda la sección: solo esas partes.
4. **Primer repaso del mazo (15 min).** `Git: Pull` → **Review flashcards from all notes** → haz clic en `s1-plataforma-net`. Te salen las 38 (17 de la teoría, 9 de errores, 12 de dudas). Contesta en voz alta cada una antes de voltearla. `Git: Commit-and-sync` al terminar.

No abras `lab-primer-proyecto`: todavía no haces ese laboratorio. Y `0-1-datos` ábrelo solo si ya estudiaste la clase de 0.1 Datos.

Los días siguientes

Fecha	Día	Qué haces
Vie 16 oct	2	Tarjetas que te toquen + cuenta en voz alta "El viaje de Program.cs" (3 min)
Dom 18 oct	4	En la revisión semanal: tarjetas + Feynman de los 2 conceptos que más te cuesten
Jue 22 oct	8	**Segunda evaluación:** las 8 preguntas de repaso de la sección 1, por escrito y sin mirar. Meta: pasar de 6.9 a 8 o más
Sáb 31 oct	17	Entra en el primer examen acumulado
Desde ahí	—	Solo tarjetas, cuando el plugin te las muestre

La rutina

Cada día de estudio (5–10 min, al inicio de la sesión)

Las clases ya empiezan con un bloque de repaso. Úsalo así:

1. Abre el repaso del plugin y haz **todas las tarjetas que te toquen hoy**. Al principio serán pocas; con el tiempo, unas 15–25 por día.
2. Si una tarjeta te falla, no la estudies en ese momento: el plugin te la volverá a mostrar pronto.
3. Si el repaso te toma más de 10 minutos, se te acumularon tarjetas: no agregues tarjetas nuevas hasta ponerte al día.

Cada domingo (dentro de los 15 min de revisión semanal)

1. Elige **2 conceptos al azar** de semanas anteriores y explícalos en voz alta 2 minutos cada uno. Grábate de vez en cuando.
2. Revisa en 14 Evaluaciones tus conceptos débiles y comprueba si ya subieron de nivel.
3. Escribe tarjetas nuevas para lo que aprendiste en la semana y no tenga tarjeta todavía.

Cada fin de mes: examen acumulado (30 min)

Al inicio de la sesión del último sábado del mes: **31 oct, 28 nov y 26 dic** (el del 26 dic coincide con el repaso final del plan).

1. Le pides a Claude un examen de 15–20 preguntas **mezcladas** de todo lo visto hasta ese día, sin decirle de qué tema empezar. Mezclar obliga a tu cerebro a elegir el concepto correcto, como en una entrevista.
2. Contestas sin mirar, por escrito o en voz alta.
3. Claude te califica y el resultado entra a 14 Evaluaciones. Cada error se vuelve tarjeta de `#flashcards/errores`.

Después del plan (desde enero de 2027)

- **10 minutos diarios de tarjetas**, aunque ya no estudies temas nuevos. Las tarjetas que ya dominas aparecerán cada varios meses; casi no cuesta tiempo.
- **Un examen acumulado al mes** hasta tener empleo nuevo. En la semana previa a cada entrevista, uno extra.

Registro de unidades

Una fila por unidad. Se actualiza cuando estudias una unidad nueva y después de cada evaluación.

Unidad	Mazo	Tarjetas	Estudiada	Primer repaso	Evaluaciones	Estado
Teoría C#, sección 1	`csharp/s1-plataforma-net`	38	13 y 14 oct	15 oct (planeado)	13 oct: 6.9 · 22 oct: pendiente (atrasada)	En repaso
Teoría C#, sección 2	`csharp/s2-anatomia-proyecto`	38 (21 + 11 dudas + 6 errores)	19 oct	20 oct	19 oct: 5.3 · 27 oct: pendiente	Día 0. Siguen: 20, 21, 23 y 27 oct
Teoría C#, sección 3	`csharp/s3-tipos`	26	23 oct	24 oct	31 oct: pendiente	Día 0. Siguen: 24, 25, 27 y 31 oct
Teoría C#, sección 4	`csharp/s4-control-flujo`	22	—	—	—	Sin estudiar
Teoría C#, sección 5	`csharp/s5-double-decimal`	14	—	—	—	Sin estudiar
Teoría C#, sección 6	`csharp/s6-metodos`	18	—	—	—	Sin estudiar
Teoría C#, sección 7	`csharp/s7-convenciones`	12	—	—	—	Sin estudiar
Clase 0.1 Datos	`fundamentos/0-1-datos`	17	Por confirmar	—	—	Por confirmar si ya se estudió
Laboratorio 6 oct	`csharp/lab-primer-proyecto`	27 (15 + 4 errores + 8 dudas)	24 oct (en curso: laboratorios 0 a 2)	25 oct	24 oct, lab 2: 7.5	Falta el laboratorio 3 (en curso)

Historia para recordar el recorrido completo

Los conceptos aislados se olvidan; una historia se recuerda. Esta une todo lo de la sección 1 de 10 Teoria de CSharp. Léela, ciérrala y cuéntala tú con tus palabras.

El viaje de Program.cs

> Program.cs es una carta escrita en español (C#) que necesita llegar a una fábrica donde solo se habla un idioma de máquinas (código máquina).
> Primero pasa por la **oficina de traducción**, que es el **SDK**. Ahí el **jefe de obra, MSBuild**, lee la orden de trabajo (el `.csproj`) y llama a **Roslyn, el traductor**. Roslyn revisa la carta: si encuentra una frase sin sentido, como guardar texto en un `int`, la regresa y no traduce nada (**error de compilación**). Si todo está bien, la traduce a un **idioma universal**, el **IL**, y la mete en una **caja sellada**: el **ensamblado** `.dll`. La caja lleva una **etiqueta** (el **manifiesto**) con su nombre, versión y la lista de otras cajas que necesita, y un **inventario** de todo lo que contiene (los **metadatos**). En la tapa trae escrito **MZ**, como todas las cajas de Windows.
> Cuando alguien quiere usar la caja, llega primero la **recepcionista**, el **host** `dotnet.exe`. Lee la nota pegada (`runtimeconfig.json`), ve qué versión de fábrica necesita y llama al **gerente de la fábrica, el CLR**. El gerente prepara la **bodega** (el **heap**) y contrata al **personal de limpieza** (el **recolector de basura**). Abre la caja, busca la primera instrucción (`Main`) y, cada vez que se necesita una instrucción por primera vez, llama al **intérprete simultáneo, el JIT**, que la traduce al idioma exacto de las máquinas de esa fábrica (x64 o ARM64). Si algo explota (una **excepción**), el gerente busca quién se hace cargo; si nadie, cierra la fábrica e imprime el reporte de lo ocurrido (el **stack trace**).
> En DiSí, la fábrica está dentro de un edificio con **recepción propia**, **IIS**, que recibe a los clientes de internet y los manda al piso correcto (`w3wp.exe`), donde trabaja el gerente.

Cómo usarla: cuéntala completa en voz alta una vez por semana durante el primer mes. Si te trabas en una parte, esa es la que no entiendes todavía.

Cómo medir si funciona

Señal	Qué significa	Qué hacer
Más del 85% de tarjetas en Good o Easy (Bien o Fácil)	El sistema funciona	Seguir igual
Muchas tarjetas en Again o Hard	Estás memorizando sin entender	Volver a la nota y explicarla en voz alta antes de seguir
La misma tarjeta falla 3 veces	La tarjeta está mal hecha	Reescribirla o dividirla
El repaso diario pasa de 15 min	Agregas tarjetas más rápido de lo que las aprendes	Una semana sin tarjetas nuevas
Las calificaciones de 14 Evaluaciones suben mes con mes	Retención a largo plazo	Ese es el objetivo

---

Cosas que parecen estudiar pero no sirven

- **Releer y subrayar.** Se siente productivo y casi no deja nada.
- **Estudiar todo un día antes.** Sirve para mañana, no para dentro de seis meses.
- **Ver videos sin pausar a explicarte lo que viste.**

- **Desvelarte.** La memoria se consolida mientras duermes; dormir poco borra parte de lo que estudiaste ese día.